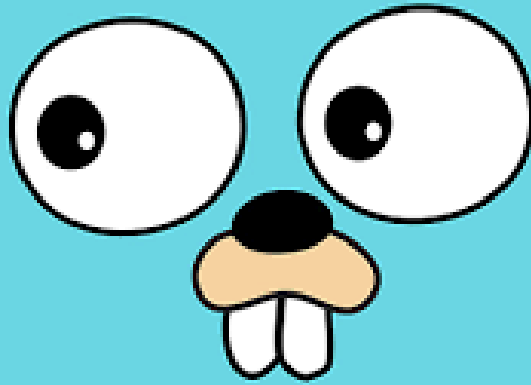




Select and Sync



Discussion Points

- The select statement
- Synchronization primitives: WaitGroup, Mutex, RWMutex
- Atomic package





The select statement

What is **Select** statement?

Select statement in Golang is possibility to *Multiplexing on multiple channel*

To work with multiple channels, we use the **select** statement. Similar to a *switch* statement, select allows you to handle multiple channel operations, such as **reads, writes, or a combination**, making it easier to manage communication between multiple channels.

```
select {  
  case <-c1:  
    fmt.Println("Code to execute when a value is received from c1")  
  case data := <-c2:  
    fmt.Println("Code to execute when a value is received from c2", data)  
  case c3 <- z:  
    fmt.Println("Code to execute when z is sent to c3")  
  default:  
    fmt.Println("Optional default, if none of the other operations can proceed")  
}
```

Using the default option in a select statement is useful for non-blocking sends and receives. The default case in a select is executed when no channels are available to send or receive data. To avoid deadlock, we can use default case.





The select statement

What is Empty select?

Similar to an empty `for{} loop`, an empty `select{} is also valid but has a caveat: since select waits for one of its cases to unblock, having no cases means the main goroutine will block indefinitely, causing a deadlock.`

In the given program, the select statement blocks the main goroutine, allowing the scheduler to run the available service goroutine. However, after service completes, the scheduler attempts to schedule another goroutine but finds none available, while the main goroutine remains blocked, ultimately causing a deadlock.

```
func main() {  
    fmt.Println("main() started")  
  
    go func() {  
        fmt.Println("Hello from service!")  
    }()  
  
    select {}  
  
    fmt.Println("main() stopped")  
}
```





The select statement

Channels conclusion

- *Avoid Global Channels*: Instead of global channels, **prefer passing channels as function arguments**. This promotes better encapsulation.
- *Detecting Deadlocks*: Go's runtime can **detect certain deadlocks** and will panic if one is faced.
- *Avoid 'Busy Waiting'*: Instead of frequently polling a channel, **utilize select or range** to wait for values efficiently.
- If necessary, only use memory access synchronization (**mutexes**).

Go channels are highly recommended to use in Go applications when dealing with concurrency. But if your problem can not be solved with channels, you may still use other solutions by **sync package**. This package provides low-level components like **mutexes**.





Synchronization primitives: WaitGroup, Mutex

What is WaitGroup?

In certain situations, you may need to block specific parts of your code to ensure goroutines finish executing as intended. A common use of **WaitGroup** is to block the main function, which itself runs as a goroutine, until all other goroutines have completed.

```
fmt.Println("main() started")

wg := sync.WaitGroup{}
wg.Add(1)

go func() {
    fmt.Println("Goroutine runs some logic")
    defer wg.Done()
}()

wg.Wait()

fmt.Println("main() end")
```

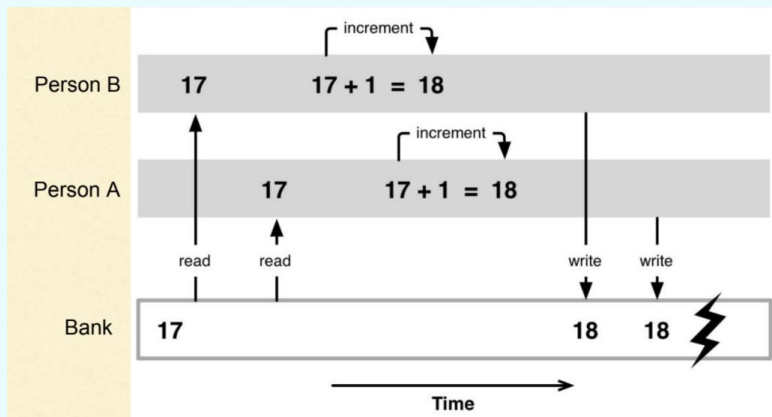




Synchronization primitives: WaitGroup, Mutex

What is Race Condition?

In certain situations, you may need to block specific parts of your code to ensure goroutines finish executing as intended. A common use of **WaitGroup** is to block the main function, which itself runs as a goroutine, until all other goroutines have completed.





Synchronization primitives: WaitGroup, Mutex

What is **Mutex**?

A **Mutex** is used to provide a locking mechanism that ensures only one goroutine can execute a critical section of code at a time, preventing race conditions.

Available in the sync package, Mutex has two methods: *Lock* and *Unlock*. Any code between these two calls is executed by only one goroutine, ensuring thread safety.

```
var m sync.Mutex
m.Lock()
// Block of code
m.Unlock()
```





Synchronization primitives: WaitGroup, Mutex

What is Reader/Writer Mutex?

Sometimes, it's ok for data to have concurrent reads, as long as the writes stay atomic.

In this case, we can use a **sync.RWMutex** type, which has different locks for reading and writing to data:

```
func main() {  
    m := map[int]int{}  
  
    mux := &sync.RWMutex{}  
  
    go writeLoop(m, mux)  
    go readLoop(m, mux)  
    go readLoop(m, mux)  
    go readLoop(m, mux)  
    go readLoop(m, mux)  
  
    time.Sleep(10 * time.Millisecond)  
  
    fmt.Println(m)  
}
```

```
func writeLoop(m map[int]int, mux *sync.RWMutex) {  
    for {  
        for i := 0; i < 5; i++ {  
            mux.Lock()  
            m[i] = i  
            mux.Unlock()  
        }  
    }  
}  
  
func readLoop(m map[int]int, mux *sync.RWMutex) {  
    for {  
        mux.RLock()  
        for k, v := range m {  
            fmt.Println(k, "-", v)  
        }  
        mux.RUnlock()  
    }  
}
```

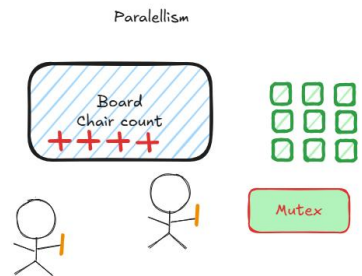
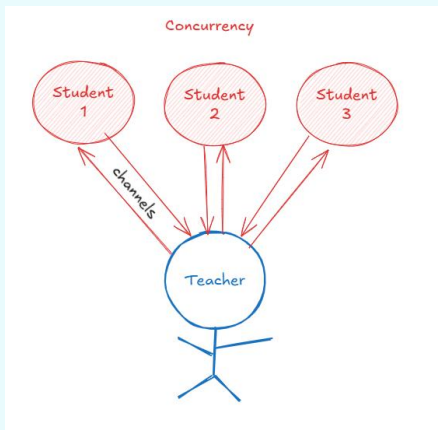




Synchronization primitives: WaitGroup, Mutex

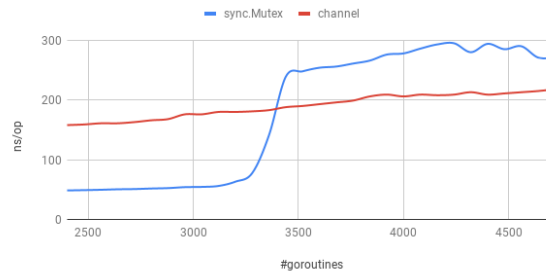
Mutex vs Channels?

In general, we need **mutexes** for **parallel goroutines** and **channels** for **concurrent ones**.



sync.Mutex performance

bench: https://play.golang.org/p/f_u9jHBq_Jc





Synchronization primitives: WaitGroup, Mutex

Atomic package?

Package atomic provides low-level atomic memory primitives useful for implementing synchronization algorithms.

These functions require great care to be used correctly. Except for special, low-level applications, synchronization is better done with channels or the facilities of the sync package. Share memory by communicating; don't communicate by sharing memory.



<https://pkg.go.dev/sync/atomic>



Thank you.

Questions?

- The select statement
- Synchronization primitives: WaitGroup, Mutex, RWMutex
- Atomic package

“Learn from yesterday, live for today, hope for tomorrow. The important thing is not to stop questioning.”

- **Albert Einstein**

